Why Public vs Private Variables?

- Game Analogy: Player name is visible to everyone (public).
- But secret cheat code or health data should be hidden (private).
- Finance Analogy: Account owner is public, but balance is private.
- We need control to prevent misuse or accidental modification.

Definition: Public, Private, Class Variables

- Public: Accessible anywhere $\rightarrow$ self.name
- Private: Defined with __ (double underscore), restricted access
- Class Variables: Shared across all instances of a class

Problem 1: Player with Public and Private Variables

• Create a Player with name (public) and __health (private). Show access difference.

Solution 1: Public vs Private

```
class Player:
    game_name = "BattleZone"  # class variable

    def __init__(self, name, health=100):
        self.name = name        # public
        self.__health = health  # private

    def get_health(self):
        return self.__health

p1 = Player("Ali")
print(p1.name)         # Ali
print(p1.get_health()) # 100
print(Player.game_name)
```

Problem 2: Count Total Accounts

- Track how many BankAccount objects have been created using a class variable.

Solution 2: Class Variable Shared

```
class BankAccount:
    account_count = 0

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance
        BankAccount.account_count += 1

    def get_balance(self):
        return self.__balance

a1 = BankAccount("Ali", 2000)
a2 = BankAccount("Sara", 3000)
print("Total Accounts:", BankAccount.account_count)
```

Why Use @property?

- Game Analogy: Health cannot exceed 100 → must control setting.
- Finance Analogy: Loan rate must stay between 1%–20%.
- @property lets us define safe getter and setter methods.

Problem 1: Loan with Rate Validation

• Create Loan with __rate. Use @property to ensure rate 1–20.

Solution 1: Loan with Property

```
class Loan:
    def __init__(self, principal, rate):
        self.principal = principal
        self.__rate = rate

    @property
    def rate(self):
        return self.__rate

    @rate.setter
    def rate(self, value):
        if 1 <= value <= 20:
            self.__rate = value
        else:
            raise ValueError("Rate must be between 1 and 20")

loan = Loan(100000, 12)
loan.rate = 15
print(loan.rate)
```

Problem 2: Prevent Negative Balance

- Use @property setter to prevent negative balance.

Solution 2: Account Balance

```
class Account:
    def __init__(self, balance=0):
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, value):
        if value >= 0:
            self.__balance = value
        else:
            raise ValueError("Balance cannot be negative")

acc = Account(5000)
acc.balance = 6000
print(acc.balance)
```

Why CRUD in Classes?

- Game Analogy: Manage inventory → add, remove, update weapons.
- Finance Analogy: Customer records → create, update, delete.
- CRUD = Create, Read, Update, Delete.

Problem 1: Customer Manager

- Manage customers with CRUD operations.

Solution 1: Customer Manager

```
class CustomerManager:
    def __init__(self):
        self.customers = {}

    def create(self, id, name):
        self.customers[id] = name

    def read(self, id):
        return self.customers.get(id, "Not Found")

    def update(self, id, new_name):
        if id in self.customers:
            self.customers[id] = new_name

    def delete(self, id):
        if id in self.customers:
            del self.customers[id]

cm = CustomerManager()
cm.create("C001", "Ali")
cm.update("C001", "Ali Raza")
print(cm.read("C001"))
```

Problem 2: Portfolio Manager

- Manage stocks with add, update, delete operations.

Solution 2: Portfolio

```
class Portfolio:
    def __init__(self):
        self.stocks = {}

    def add_stock(self, symbol, price):
        self.stocks[symbol] = price

    def update_stock(self, symbol, price):
        if symbol in self.stocks:
            self.stocks[symbol] = price

    def remove_stock(self, symbol):
        self.stocks.pop(symbol, None)

    def view_all(self):
        return self.stocks

p = Portfolio()
p.add_stock("AAPL", 180)
p.add_stock("TSLA", 250)
p.update_stock("TSLA", 260)
print(p.view_all())
```

Why Instance vs Class Methods?

- Game Analogy: Player jump → instance method, total players → class method.
- Finance Analogy: Loan repayment → instance, total loans → class method.
- Static methods = independent utilities.

Problem 1: Total Loans with Class Method

- Count total loans using a class method.

Solution 1: Class Method

```
class Loan:
    loan_count = 0

    def __init__(self, principal):
        self.principal = principal
        Loan.loan_count += 1

    @classmethod
    def get_total_loans(cls):
        return cls.loan_count

l1 = Loan(100000)
l2 = Loan(200000)
print(Loan.get_total_loans())
```

Problem 2: Validate Rate with Static Method

- Write static method validate_rate(rate).

Solution 2: Static Method

```
class Loan:
    @staticmethod
    def validate_rate(rate):
        return 0 < rate <= 20

print(Loan.validate_rate(15))  # True
print(Loan.validate_rate(25))  # False
```

Why Lists & Tuples as Members?

- Game Analogy: Player has many weapons (list), each weapon has fixed attributes (tuple).
- Finance Analogy: Customer has many accounts (list), loan has repayment schedule (list of tuples).
- Useful for storing multiple related values in an object.

Problem 1: Customer with Multiple Accounts

- Store multiple accounts in a list and calculate total balance.

Solution 1: Customer with List

```
class Customer:
    def __init__(self, name):
        self.name = name
        self.accounts = []  # list of balances

    def add_account(self, balance):
        self.accounts.append(balance)

    def total_balance(self):
        return sum(self.accounts)

c = Customer("Ali")
c.add_account(5000)
c.add_account(7000)
print(c.total_balance())
```

Problem 2: Loan Repayment Schedule

• Loan stores repayment schedule as list of (year, emi) tuples.

Solution 2: Loan Schedule

```
class Loan:
    def __init__(self, principal, years, emi):
        self.schedule = []
        for year in range(1, years+1):
            self.schedule.append((year, emi))

    def get_schedule(self):
        return self.schedule

loan = Loan(100000, 3, 35000)
print(loan.get_schedule())
```

Wrap-Up

- Public, Private, and Class Variables → control access
- @property → safe attribute access
- CRUD → managing collections
- Instance vs Class vs Static Methods → different scopes
- Lists/Tuples as Members → organize multiple values in one object